

Повторное использование KV-кэша для долго работающих агентов

Можно ли сжимать данные, добавлять новый контекст и снова сжимать, сохраняя фиксированный объем памяти по мере накопления информации?



Чарльз О'Нил (Базеттен)

Кодирующий агент накапливает результаты работы инструментов в течение нескольких часов, а клинический ассистент обрабатывает записи о пациентах по одной. В обоих случаях контекст неограниченно расширяется, в то время как объем памяти остается неизменным.

Стандартные решения (усечение, обобщение, извлечение) в каждом случае чем-то жертвуют. При усечении предполагается, что старый контекст можно не учитывать. При обобщении происходят потери, которые невозможно предсказать: значение, которое казалось несущественным во время сжатия, оказывается ответом на более поздний вопрос. Извлечение требует знания того, что является релевантным, еще до получения запроса.

Вместо работы с текстом Attention Matching ([Цвайгер и др., 2026](#)) сжимает кэш пар «ключ-значение» модели непосредственно в латентном пространстве, заменяя T кэшированных пар «ключ-значение» на $t \ll T$ подобранных замен, которые обеспечивают такое же поведение модели. После однократного применения достигается сжатие в 50 раз с минимальной потерей качества за считанные секунды.

Но однократного сжатия недостаточно для растущего контекста. Можно ли сжимать данные, добавлять новый контекст и снова сжимать, сохраняя фиксированный объем памяти по мере накопления информации?

Краткое сопоставление внимания

Когда механизм внимания обрабатывает два объединенных KV-блока, он разбивает их на смесь локальных результатов внимания каждого блока, взвешенных в соответствии с долей каждого блока в общей ненормированной массе. Это означает, что сжатие сводится к двум локальным задачам, которые не зависят от последующих токенов: сопоставить результат внимания блока и его массу внимания.

Алгоритм решает эти задачи последовательно, для каждого слоя и каждой KV-головки. Сначала выбираются t ключей из исходного набора T по оценке внимания. Затем с помощью метода неотрицательных наименьших квадратов подбирается скалярное смещение $\beta \in \mathbb{R}^t$, чтобы перевзвешенная масса компактных ключей соответствовала исходной. Без β сжатый блок будет систематически недоучитываться при любой последующей конкатенации. В-третьих, подберите значения замены $C_v \in \mathbb{R}^{t \times d}$ с помощью обычного метода наименьших квадратов таким образом, чтобы выходные данные компактного блока соответствовали исходным при новых весовых коэффициентах.

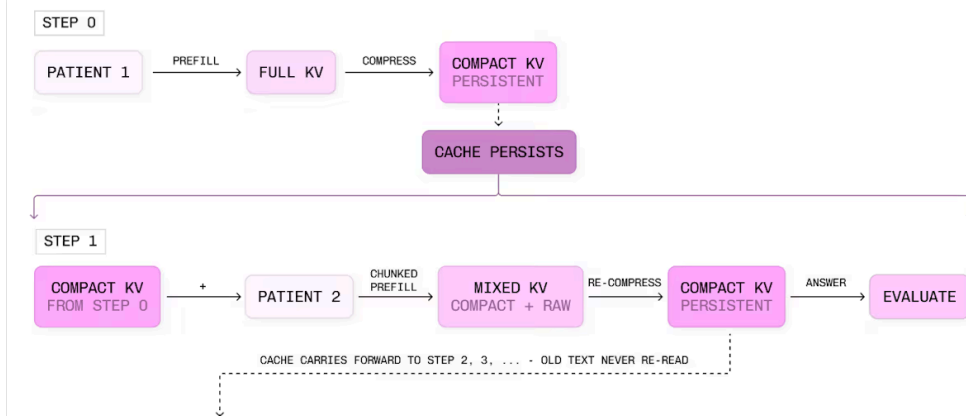
Для дальнейшего изложения важны две детали. Подбор значений решает задачу $\min_{C_v} \|XC_v - Y\|_F^2$, где каждая строка X представляет собой вектор весовых коэффициентов внимания по компактным ключам для одного эталонного запроса, а Y содержит соответствующие целевые результаты внимания. Это стандартная регрессия, и ее качество зависит от условий X . А справочные запросы Q_{ref} , прокси для будущих запросов, формируются на основе прямого прохода по самому контексту, что позволяет создавать широкие, восстанавливающие паттерны внимания, а не четкие паттерны поиска.

Эксперимент: Инкрементное сжатие

Мы использовали Qwen3-4B на [LongHealth](#), клиническом бенчмарке для контроля качества, с медицинскими картами 20 вымышленных пациентов. На каждого пациента приходится около 12 000 токенов подробных клинических записей и 20 вопросов с несколькими вариантами ответов, проверяющих фактическую информацию: имена пациентов, результаты анализов, дозировки лекарств и порядок их приема. Мы смоделировали последовательный прием пациентов: 5 пациентов приходят по очереди, и после каждого прихода мы сжимаем накопленный контекст до фиксированного объема памяти, прежде чем отвечать на вопросы обо всех пациентах, которых мы уже приняли. Результаты усреднены по 4 контекстным группам (20 пациентов, 400 вопросов на каждом этапе).

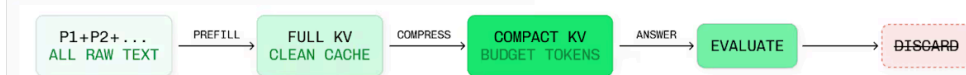
True re-compaction

Compress once, then append new data onto the persistent cache and re-compress. Old text is never re-read.



Fresh compaction

At each step, re-prefill all accumulated raw text from scratch. Build a clean KV cache, compress once, answer, discard



При свежем сжатии на каждом этапе создается полный кэш из исходного текста — надежный, но дорогостоящий метод, линейно масштабируемый в зависимости от длины контекста. При повторном сжатии считываются только новые данные, что позволяет поддерживать постоянную стоимость на каждом этапе, но при этом сжимаются уже сжатые представления, что приводит к накоплению ошибок аппроксимации.

Свежее сжатие: на каждом этапе заново заполняем весь накопленный текст с нуля, а затем за один раз сжимаем чистый кэш. Ни одно приближение не влияет на последующее приближение. Мы тестируем этот метод как при фрагментарном сжатии (независимая подгонка для каждого пациента с последующим объединением результатов), так и при монолитном сжатии (единая подгонка методом наименьших квадратов для всей последовательности).

Истинное повторное сжатие: постоянный кэш, который постепенно увеличивается. Токены каждого нового пациента добавляются в существующий (ранее сжатый) кэш с помощью предварительного заполнения блоками, после чего весь кэш повторно сжимается до нужного размера. Именно так поступил бы настоящий агент: он никогда не перечитывает старый текст. Мы протестировали этот метод с монолитным сжатием при бюджете в 4096 токенов.

Finding 1: Local fitting is a structural requirement

Our first result had nothing to do with re-compaction. At a 12,288-token budget, fresh monolithic compaction (a single least-squares fit over the entire multi-patient sequence) scored 30.8% at step 1 (24k $\rightarrow$ 12k, just 2 $\times$ compression). At the same budget, fresh chunked compaction scored 79.3%.

The 49% gap comes from the structure of attention on concatenated independent documents. When the model processes a token inside Patient 1's chemotherapy notes, its attention is overwhelmingly local: it attends to the dosage, the cycle timing, and the protocol name. It has no reason to attend to Patient 2's renal imaging. The result is an approximately block-diagonal attention matrix: dense within each patient, near-zero between patients.

This structure makes global least-squares fitting ill-posed. Each row of the design matrix X is a compact attention weight vector for one reference query. Queries from Patient 1 load on Patient 1's columns; queries from Patient 2 load on Patient 2's columns. The rows cluster into nearly orthogonal subspaces. A single C_v must serve both, but because the subspaces barely overlap, fitting one patient's queries tells you almost nothing about the other's.

We tested the obvious alternative explanation: that the system was underdetermined. A query budget sweep from 20,000 to 60,000 queries per KV head (5 $\times$ overdetermination relative to the number of compact keys) moved accuracy from 21.7% to 23.8%. More rows in Patient 1's subspace don't help fit Patient 2's subspace. The problem is geometric, not statistical.

Chunked compaction fixes this by splitting at patient boundaries and fitting each chunk independently. Each sub-problem has dense, full-rank X and captures local attention structure faithfully. This is a structural requirement for any heterogeneous multi-document context, not an optimization choice.

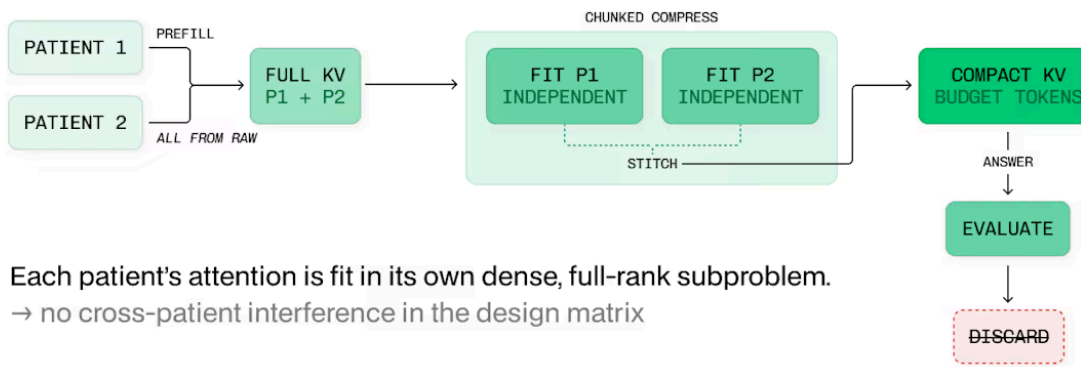
Chunked fresh compaction

At each step, re-prefill all accumulated raw text from scratch. Compress each patient independently via chunked fitting, then stitch. Discard after answering.

STEP 0 - ONE PATIENT



STEP 1 - TWO PATIENTS, FRESH FROM RAW TEXT



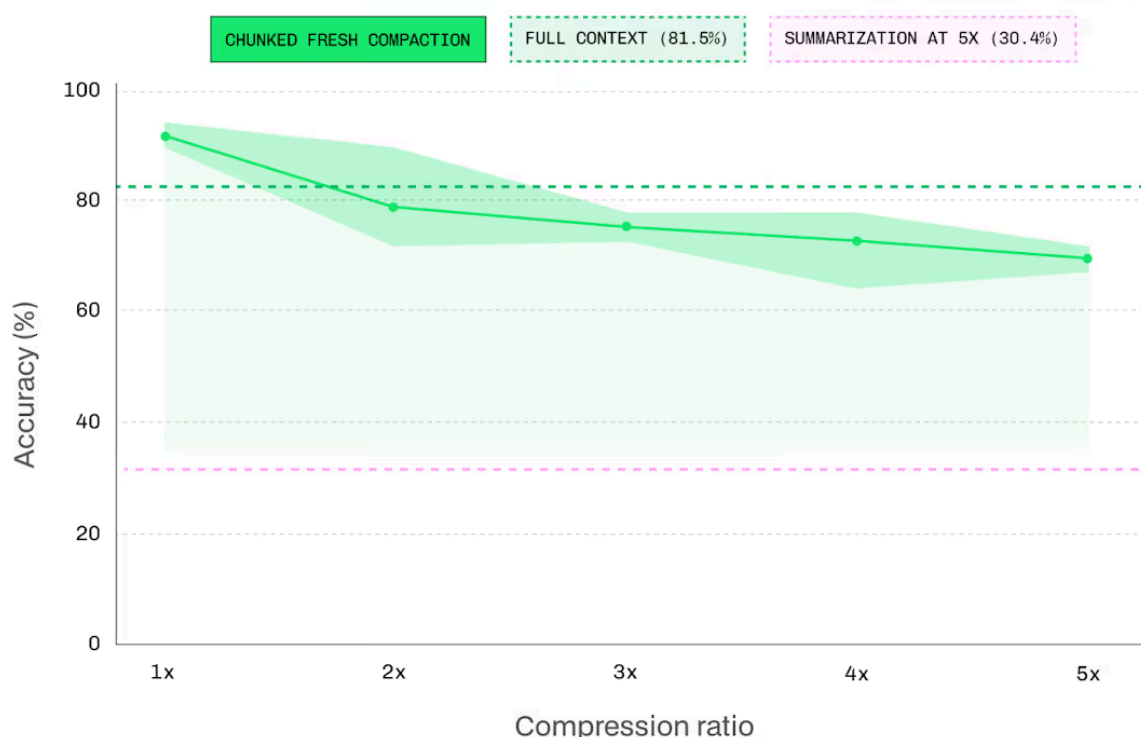
Chunked fresh compaction shares Fresh Compaction's data flow ie rebuild the full cache from raw text at every step, discard after answering. Instead of a single monolithic least-squares solve over the entire concatenated sequence, chunked compaction fits each patient independently, then stitches the results. Each sub-problem has a dense, well-conditioned design matrix.

Finding 2: Fresh compression degrades gracefully

With chunked compaction, fresh-from-text compression shows clean, predictable degradation. At a 12,288-token budget with chunked compaction (the configuration we call "Chunked fresh compaction": fresh prefill and compress at each step, using article boundaries for chunk-local fitting):

Step	Patients	Compression	Chunked fresh compaction	No Context	Full Context
0	1	1×	90.7%	34.0%	81.5%
1	2	~2×	79.3%	—	—
2	3	~3×	73.7%	—	—
3	4	~4×	69.4%	—	—
4	5	~5×	65.6%	—	—

QA accuracy vs compression ratio (12k budget)



With chunked compaction, fresh-from-text compression shows clean, predictable degradation – better than no and full context compaction.

The marginal cost of each additional increment of compression decreases: 11.4pp for the first 2x, then 5.6, 4.3, 3.8 for each subsequent 1x. Roughly logarithmic, with no phase transition or sudden collapse. At step 0, chunked fresh compaction actually *exceeds* the full-context accuracy (90.7% vs 81.5%), likely because compaction acts as a form of denoising.

At the same 12k budget, text summarization scored 30.4%, barely above the 34.0% no-context baseline. Attention matching (AM) preserves the kind of detail that summarization discards: at 5x compression, the model can still recall specific lab values, medication dosages, and temporal ordering across five patients' records.

Where does the 25% gap between uncompressed and 5x come from? Partly from information genuinely lost at high compression. But likely also from a mismatch between reference query geometry and test-time query geometry. The reference queries come from context-prefill: the model reconstructing what it just read, producing broad, diffuse attention patterns. LongHealth questions demand needle-in-a-haystack retrieval, nearly all attention on a handful of specific tokens. The fitted C_v was optimized for diffuse queries and may be poorly calibrated for sharp ones.

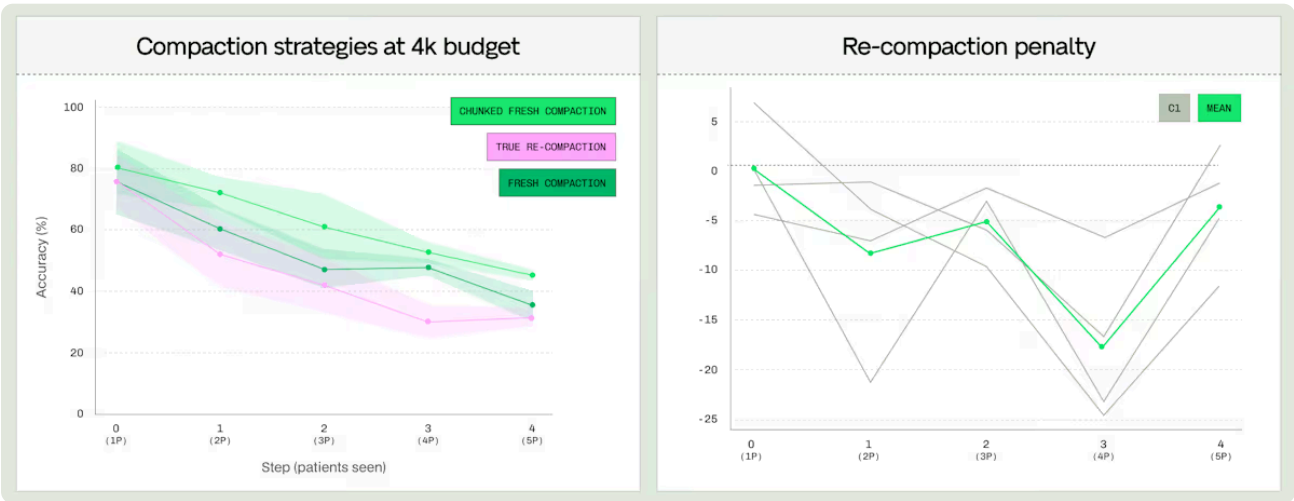
The original paper shows that self-study reference queries, which produce more targeted attention patterns, consistently improve compaction quality; we used only context-prefill.

Finding 3: The cost of re-compression

With local fitting established as a structural requirement and fresh compression characterized, we can isolate the question we started with: what happens when you compress the already-compressed?

We ran all three compaction strategies at a matched 4,096-token budget (aggressive compression, 3–15× across steps), across 4 context groups:

Step	Patients	Compression	Chunked fresh compaction	Fresh compaction	True re-compaction
0	1	~3×	80.4%	77.6%	77.8%
1	2	~6×	72.1%	61.5%	57.4%
2	3	~9×	60.9%	47.2%	42.9%
3	4	~12×	52.8%	46.4%	30.5%
4	5	~15×	45.1%	34.8%	31.2%



The compression cost is the largest component and largely unavoidable at 12× ratios. The local fitting penalty is moderate and fixable by using chunked compaction. The re-compaction penalty is the open problem.

The three lines decompose the total accuracy loss into distinct components. At step 3 (4 patients, ~12× compression), starting from the full-context baseline of 81.3%:

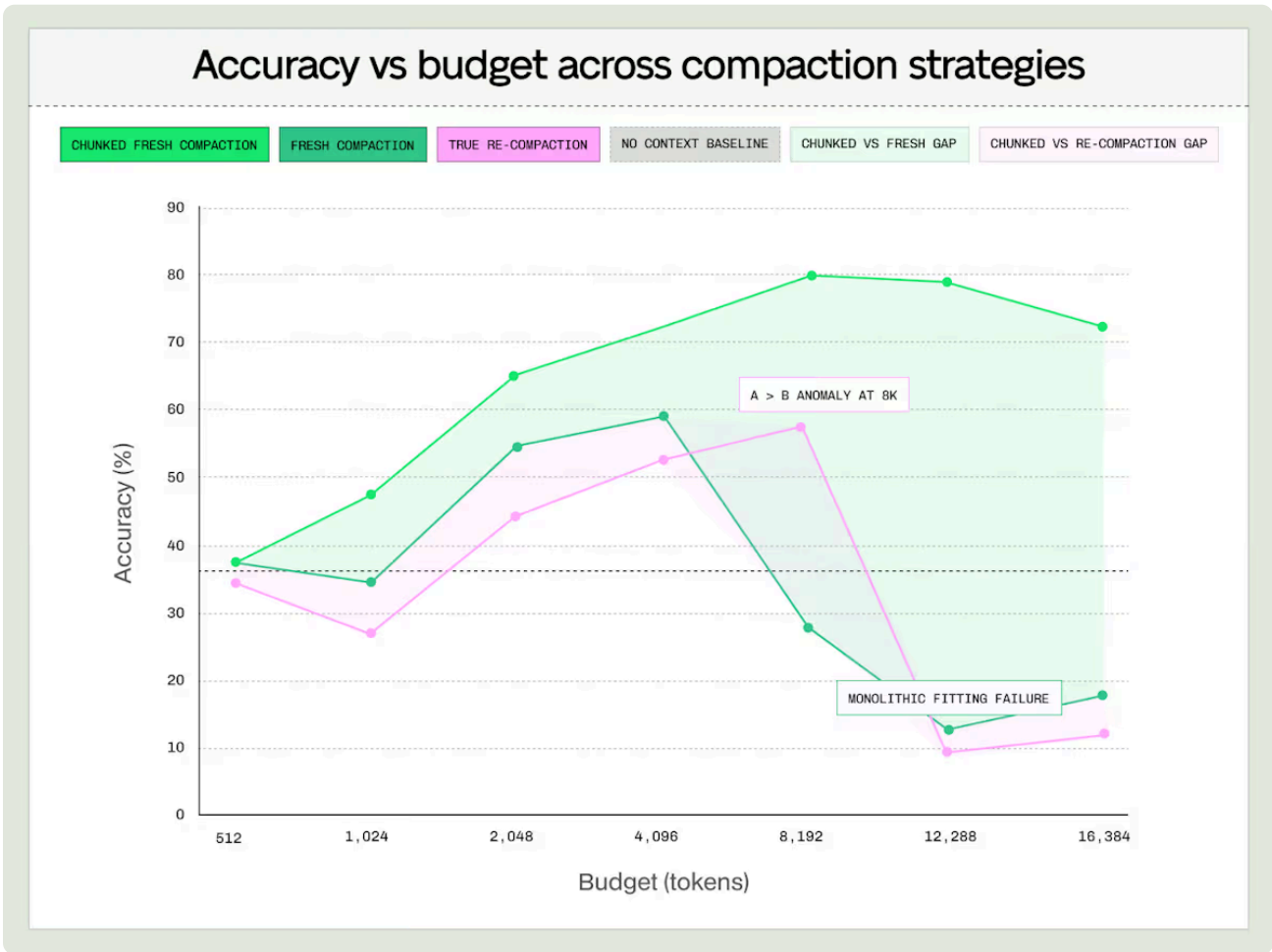
- **Chunked fresh compaction** (chunked, fresh): 52.8%, a 28.5% cost from compression itself.
- **Fresh compaction**: 46.4%, an additional 6.4% from fitting globally instead of locally.
- **True re-compaction**: 30.5%, a further 15.9% from compressing the already-compressed.

The compression cost is the largest component and largely unavoidable at 12× ratios. The local fitting penalty is moderate and fixable by using chunked compaction. The re-compaction penalty is the open problem.

How the penalties scale with compression ratio

To map these penalties more precisely, we ran all three strategies at seven budgets from 512 to 16,384 tokens, evaluating on two patients (~24k raw tokens at step 1, a single re-compaction event):

Budget	Compression	Chunked fresh compaction	Fresh compaction	True re-compaction	Re-compact penalty (A–B)
512	~48x	37.6%	37.4%	33.4%	–4.0pp
1,024	~24x	46.9%	35.0%	27.3%	–7.7pp
2,048	~12x	64.9%	55.4%	44.5%	–10.9pp
4,096	~6x	71.5%	58.6%	52.9%	–5.7pp
8,192	~3x	80.7%	28.2%	56.3%	+28.2pp
12,288	~2x	80.1%	12.9%	9.7%	–3.3pp
16,384	~1.5x	72.5%	16.7%	12.5%	–4.2pp



The data reveals two regimes separated by a sharp transition:

- **Below 8k (compression $\geq 3\times$):** Monolithic fitting works. All three paths produce coherent output. Chunked fresh compaction leads, Fresh Compaction trails by 10–13% (the local fitting penalty), and True Re-compaction trails Fresh Compaction by a further 4–11% (the re-compaction penalty). The re-compaction penalty peaks at 2,048 tokens ($\sim 12\times$ compression), suggesting that moderate compression creates the worst conditioning for the C_v regression.
- **Above 8k (compression $\leq 3\times$):** Monolithic fitting collapses. Fresh Compaction (fresh compaction with no recursive error) drops to 12–17%, confirming this is a fitting failure, not a re-compaction problem. Chunked fresh compaction, using chunked compaction, holds at 73–80%. The 50–67% gap between Chunked fresh compaction and Fresh Compaction is entirely attributable to chunked vs monolithic fitting.

At budget=8,192, True Re-compaction (56.3%) beats Fresh Compaction (28.2%) by 28%, the opposite of the expected direction. This is because re-compaction inadvertently helps: True Re-compaction's source at step 1 is $\sim 20k$ tokens (8k compressed Patient 1 + 12k raw Patient 2), shorter than Fresh Compaction's 24k (both patients uncompressed). The monolithic fit operates on a less heterogeneous sequence, accidentally sidestepping the failure. This confirms that absolute source length, not compression ratio, is the critical variable for monolithic fitting.

All outputs remain coherent and parseable throughout. The model doesn't collapse; it just loses accuracy. But the re-compaction penalty is systematic and grows with rounds.

What drives it? The mechanism is intrinsic to recursive least-squares fitting. Each round solves:

$$C_v^* = (X^\top X)^{-1} X^\top Y$$

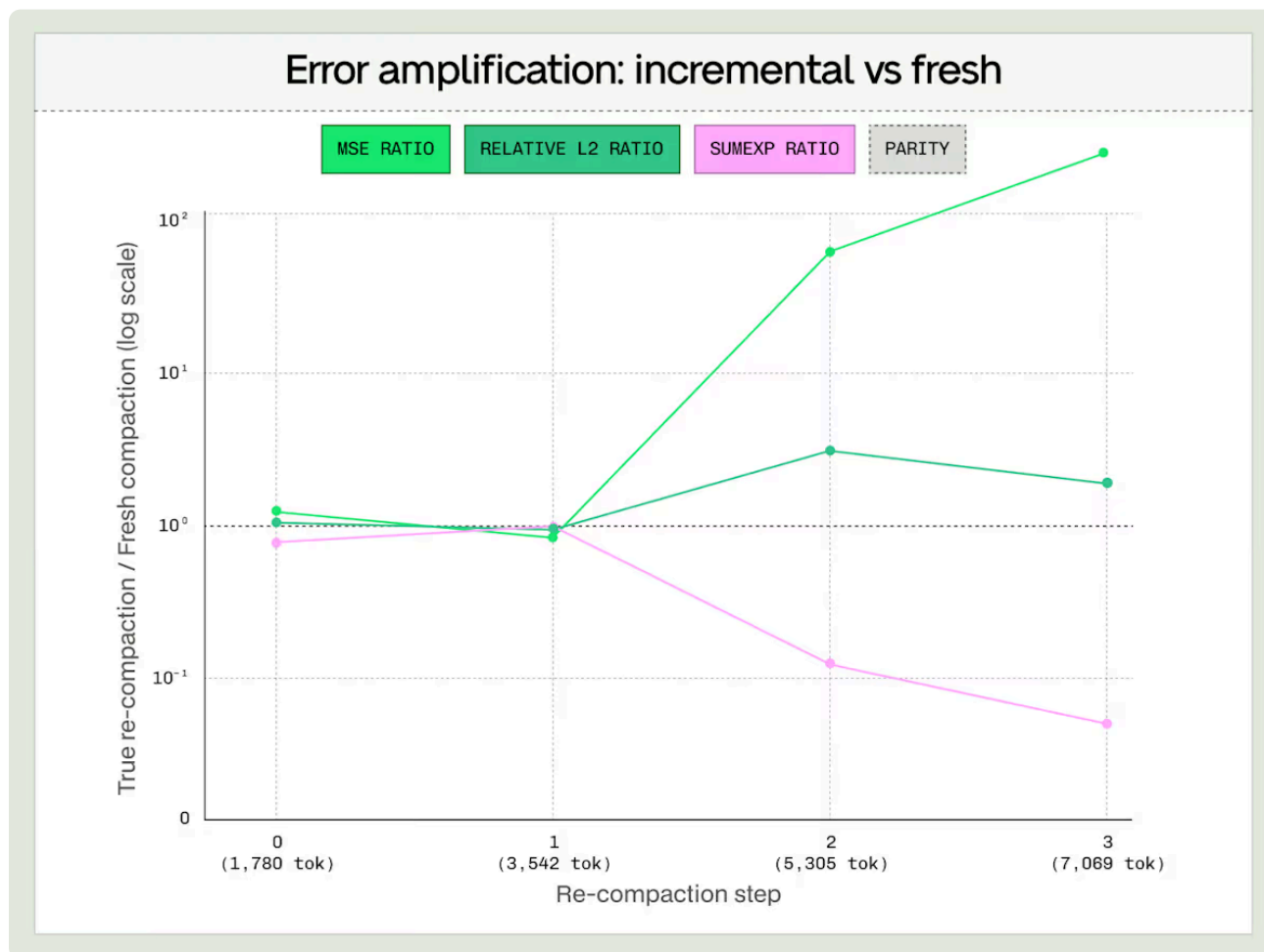
In fresh compaction, Y is always computed from a clean forward pass. The targets are exact. In re-compaction, Y is computed from the *previously compacted* cache. The targets are themselves approximations, and the error propagates through a recurrence:

$$e_{r+1} \leq \|A_r\|, e_r + \eta_r$$

where e_r is output approximation error at round r , η_r is fresh noise from the current round's compression, and $\|A_r\|$ is the local amplification factor, determined by the condition

number of the design matrix X for that round's regression.

When $\|A_r\| > 1$ on some heads, error grows geometrically across rounds. This happens when X is poorly conditioned: when the compact attention weights don't span enough of the query space, or when a few keys dominate.



Each round of lossy compression is individually reasonable, but the errors introduced by one round become the signal for the next.

The amplification is not uniform across heads. A handful of poorly-conditioned heads drive worst-case error while most remain stable.

This is the JPEG-of-a-JPEG problem. Each round of lossy compression is individually reasonable, but the errors introduced by one round become the signal for the next. The structure of AM (fitting values by least squares against attention-output targets) means the error amplification is governed by the conditioning of the attention weight matrix, which is a property of the key distribution and query distribution rather than something the algorithm can trivially control.

What this means

For practical deployment: if you can afford periodic re-prefill from raw text (say, after every few turns), you get clean compaction with no compounding penalty. Chunked fresh compaction with chunked compaction at 2–5× maintains 65–80% accuracy on dense factual QA, well above both the no-context floor and what summarization achieves at matched budget.

If you need true incremental compaction without re-reading old text, the current penalty is 4–16% at the ratios we tested. Whether this is acceptable depends on the task. For conversational agents where approximate recall is fine, it probably is. For clinical factual recall, where every lab value matters, it may not be.

Three things are likely to reduce the re-compaction penalty. Nonuniform head budgets, the original paper's most impactful component (which we didn't use), would allocate more capacity to the sensitive heads that drive worst-case error amplification. Self-study reference queries would better capture the sharp retrieval patterns exhibited by test-time queries. And regularization of the C_v fit could bound the amplification factor $|A_r|$ at the cost of per-round approximation quality.

One-shot compression is a regression problem. Repeated compression is a dynamical system. The tools you need to analyze them are different: for one-shot, it's bias-variance and approximation quality; for repeated, it's stability, amplification, and convergence. Single-round compaction already works well. The open question is how to make the recurrence stable. We'll explore this more in our future research.

For attribution in academic contexts, please cite this work as:

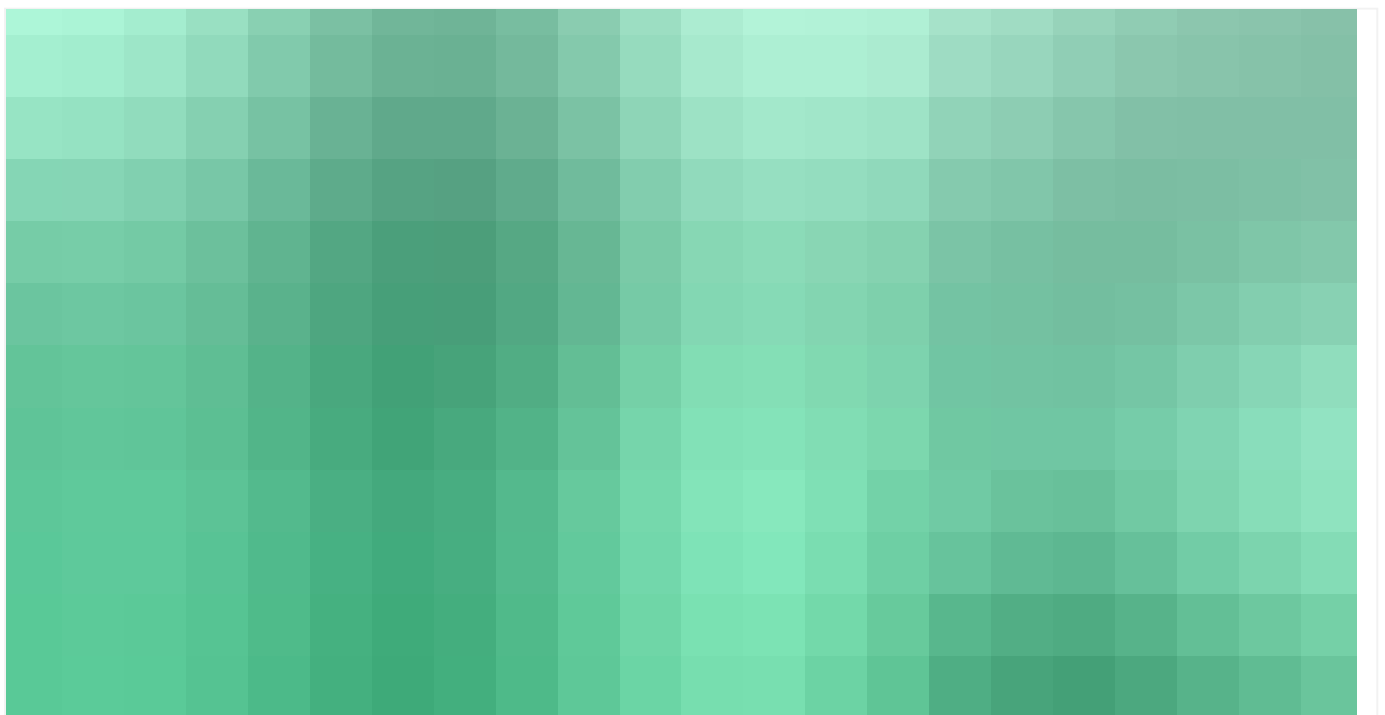
O'Neill, Charles. "Repeated KV Cache for Long-Running Agents." Baseten Research, March 5, 2026. <https://www.baseten.co/research/repeated-kv-cache-for-long-running-agents/>

BibTeX citation

```
@misc{oneill2025rgt,  
  author      = {O'Neill, Charles and Partridge, Harry and Jayasekara,  
Mudith},  
  title       = {Upweight the Strategy, Not the Tokens: Faster Training with  
Explicit Reasoning Through {RGT} (Rationale-Guided Training)},  
  year        = {2025},  
  month       = oct,  
  day         = {29},  
  howpublished = {Baseten Research},
```

```
url      = {https://www.baseten.co/research/upweight-the-strategy-not-  
the-tokens-faster-training-with-explicit-reasoning-thro/},  
note     = {Accessed: 2026-03-12}  
}
```

Other research



RESEARCH

Post-Training Science for Supervised Fine-Tuning

How the optimal learning rate and batch size move with model scale, family, and data, and whether one selection rule transfers across them.



CHARLES O'NEILL • 2 others



RESEARCH

Still: Amortized KV Cache Compaction in a Single Forward Pass

You can shrink a language model's KV cache by 200×, in a single forward pass, and it still answers correctly.



CHARLES O'NEILL • 4 others



RESEARCH

Post-training frontier legal agents with Baseten Research

Post-training a 27B open-weight model to the frontier on Harvey LAB with Baseten Research



CHARLES O'NEILL • 5 others

Explore Baseten today

START DEPLOYING

TALK TO AN ENGINEER >

Product

Dedicated Inference

Model APIs

Training

Frontier Gateway

Baseten Inference Platform

Model Runtimes

Infrastructure

Multi-cloud

Deployment options

Cloud

Self-hosted

Hybrid

Embedded engineering

Forward deployed engineers

Modalities

- Transcription
- Image Generation
- Text-to-speech
- Large language models
- Compound AI
- Embeddings

Industries

- Enterprise
- Healthcare

Developer

- Model library
- Documentation
- Changelog

Resources

- Research
- Customers
- Blog
- Guides
- Events
- Partners
- Savings calculator
- Trust Center
- About us
- Startups
- Careers
- Contact us

Popular models

- GLM 5.2
- Kimi K2.7 Code
- DeepSeek V4
- GPT OSS 120B
- Whisper Large V3
- NVIDIA Nemotron 3 Ultra
- Explore all

Legal

- Terms and Conditions
- Privacy Policy
- Service Level Agreement

● ALL SYSTEMS NORMAL



